

Auditoria Técnica

Projeto **Batch-Control** · Backlog priorizado de correções e polimentos

Data	17/08/2026
Escopo	Camadas GUI, Controller, Service, Forms, Helpers, DTOs
Status	Projeto em desenvolvimento — base saudável, backlog em aberto

Sumário executivo

A limpeza estrutural (docstrings, remoção de *dead code*, wrapper **FormField** simplificado, DTOs imutáveis, TypedDicts para dados brutos) deixou o projeto em estado saudável. O que resta é **obra em andamento** — *stubs* marcados com pass, callbacks vazios e um descasamento de chaves entre o formulário de novo produto e o construtor do DTO. Não há sujeira escondida: cada item deste relatório é rastreável a uma linha específica.

Ordem de ataque sugerida

#	Item	Prioridade
1	1.1 Descasamento de chaves NewProductFrame ↔ build_product_dto	P1
2	1.2 Ligar _on_click_save_product no controller	P1
3	1.3 Implementar update_product no service	P1
4	2.2 Zerar _edit_product_frame após destroy()	P2
5	2.1 Corrigir comparação None > 0 em delete_product	P2
6	2.3 Validação simétrica em build_edit_product_dto	P2
7	3.x Polimentos (sanitize_string, Enum de estado, etc.)	P3

P1

Prioridade 1 — Vai quebrar assim que o botão for ligado

Bugs em código já ligado ao fluxo, mas ainda não exercitados pelo usuário.

1.1

Descasamento de chaves entre NewProductFrame e build_product_dto

Onde: `src/gui/components/product_frame/new_product_frame.py` linhas 136–146 vs `src/forms/product_form.py` linhas 31–61.

O problema:

O frame devolve chaves em **português**, mas o builder acessa chaves em **inglês** e ainda referencia `raw_data["product_status"]`, que **nunca é coletado** pelo formulário.

```
# NewProductFrame.get_raw_values() devolve:
{"nome_produto": ..., "saldo_min": ..., "empresa": ...,
 "cod_sku": ..., "consumo_mensal": ...}

# build_product_dto espera:
raw_data["name"], raw_data["minimum_balance"], raw_data["product_firm"],
raw_data["product_code_chb"], raw_data["consumption_monthly"],
raw_data["product_status"] # nem existe no form!
```

Direção sugerida:

- Como `EditProductRawData` já foi definido em português, padronize as chaves do `build_product_dto` para bater com o dict do frame.
- Crie um `NewProductRawData` (`TypedDict`) simétrico ao `EditProductRawData`, e tipe o retorno de `get_raw_values` com ele.
- Defina onde `product_status` entra na criação — provavelmente default 1 (Ativo) no **service**, não no formulário.

1.2

_on_click_save_product do controller é stub

Onde: `src/gui/components/product_frame/product_controller.py` linhas 59–61.

```
def _on_click_save_product(self) -> None:
    """Callback do botão salvar no formulário de novo produto. Implementação pendente
    ."""
    pass
```

O que fazer:

- Adicionar um `get_raw_values_new_product()` ao `ProductFrame` (o atual serve só ao edit).
- Passar o dict ao `build_product_dto`.
- Capturar `ValidationError` e exibir com `ErrorWindow`.
- Chamar `self._product_service.create_product(product_dto)`.
- Capturar `DuplicateSkuError` e exibir aviso apropriado.
- Em sucesso: limpar o formulário e recarregar a tabela.

1.3 update_product do service é stub

Onde: src/service/product_service.py linhas 74–77.

```
def update_product(self, original_product_dto: ProductDTO,
                    edited_product_dto: ProductDTO) -> None:

    pass
```

Direção sugerida:

- Aproveitar diff_product_dto (já implementado em helpers.py linhas 126–152) para gerar a lista List[Tuple[campo_dto, valor_novo]].
- Converter nomes-de-DTO para nomes-de-coluna do banco antes de passar ao repository (constante compartilhada evita espalhar mapping).
- Verificar se o ProductRepository já tem um update(id, changes: dict) — se não, criar.

P2

Prioridade 2 — Bugs latentes / assimetrias

Problemas reais, mas em fluxos ainda pouco exercitados ou de recuperação silenciosa.

2.1 delete_product compara None > 0

Onde: src/service/product_service.py linhas 65–72.

```
saldo_produto = None # Futura chamada do repo de lote

if saldo_produto > 0: # TypeError garantido
    raise ProductHasBalanceError(...)
```

Direção sugerida:

- Enquanto BatchRepository não existir, inicializar como saldo_produto = 0 (permite deletar sem restrição temporariamente).
- Alternativa mais explícita: levantar NotImplementedError quando o repositório de lote for necessário, forçando implementação antes de habilitar o botão.

2.2 hide_edit_product_frame não zera o atributo após destroy()

Onde: src/gui/components/product_frame/product_frame.py linhas 170–173.

```
def hide_edit_product_frame(self) -> None:
    """Destroi o formulário de edição ativo."""
    if self._edit_product_frame is not None:
        self._edit_product_frame.destroy()
        # falta: self._edit_product_frame = None
```

Sintoma: após destruir, self._edit_product_frame continua apontando para o objeto morto. _cancel_edit_product_frame e get_raw_values acreditam que o frame ainda existe, o que pode causar crash em Tk ao acessar widgets destruídos.

Direção sugerida:

- Adicionar self._edit_product_frame = None imediatamente após o destroy().

2.3 build_edit_product_dto não valida entrada

Onde: src/forms/product_form.py linhas 63–80.

```
return replace(
    original_product_dto,
    name=sanitize_string(value=raw_data_edit["nome_produto"]),
    product_firm=sanitize_string(value=raw_data_edit["empresa"]),
    minimum_balance=int(raw_data_edit["saldo_min"]), # int("") explode
    product_status=1 if raw_data_edit["ativo"] == "Ativo" else 0,
)
```

Assimetria: o build_product_dto valida com is_number/is_valid_string, mas o build_edit_product_dto não. Se o usuário apagar o saldo_min e clicar Salvar, int("") lança ValueError sem tratamento.

Direção sugerida:

- Extrair um helper `_validate_fields(raw_data, required, number_fields)` compartilhado pelos dois builders.

P3

Prioridade 3 — Polimento e smells

Não impedem o desenvolvimento, mas melhoram legibilidade e previsibilidade.

3.1 `sanitize_string` usa `.capitalize()` em vez de `.title()`

Onde: `src/helpers/helpers.py` linhas 96–106.

```
def sanitize_string(value: str) -> str:
    return value.strip().capitalize()

# Comportamento:
# "COCA-COLA".capitalize() -> "Coca-cola"    (o 2º C vira minúsculo)
# "COCA-COLA".title()      -> "Coca-Cola"
```

Direção sugerida:

- Se a intenção é capitalizar cada palavra, trocar por `.title()`. Se era só a primeira letra do texto todo (para nomes próprios, ex. "João da Silva"), manter, mas renomear o helper para `capitalize_first` para deixar a intenção clara.

3.2 `get_state_product_frame` retorna `None` implicitamente

Onde: `src/gui/components/product_frame/product_frame.py` linhas 175–183.

```
def get_state_product_frame(self) -> Optional[str]:
    if self._state_product_frame is not None:
        return self._state_product_frame
    # cai fora sem return: devolve None implícito
```

Direção sugerida:

- Simplificar para `return self._state_product_frame` (já é `Optional[str]`).

3.3 `_state_product_frame` como string mágica

Onde: `product_frame.py` — usos de "new_product" e "edit_product" espalhados.

Direção sugerida:

- Introduzir class `ProductFrameState(Enum)`: `NEW = "new_product"`; `EDIT = "edit_product"` para eliminar risco de typo silencioso.

3.4 `get_raw_values` do `ProductFrame` tem fallback dead-code

Onde: `src/gui/components/product_frame/product_frame.py` linhas 185–199.

```
def get_raw_values(self) -> EditProductRawData:
    if self._edit_product_frame is not None:
        return self._edit_product_frame.get_raw_values()
    return EditProductRawData(
        nome_produto="", empresa="", saldo_min="", ativo=""
    )
```

O único *call site* (`_on_click_save_edit_product`) só invoca quando o frame de edição está ativo. O fallback é código defensivo sem gatilho real.

Direção sugerida:

- Trocar retorno para `Optional[EditProductRawData]` e devolver `None` — combina com o padrão de `get_product_id_to_edit`.
- Alternativa: levantar `RuntimeError` explícito ("Frame de edição não está ativo").

3.5 `diff_product_dto` ainda não é usada

Onde: `src/helpers/helpers.py` linhas 126–152.

Será usada em **1.3 (update_product)**. Se depois de implementar o service continuar sem chamada, remover para não virar *dead code*.

Apêndice — Checklist do estado atual

Aspecto	Estado	Observação
Camadas separadas (GUI/Controller/Service/Repo)	OK	Sem vazamento entre camadas.
DTOs imutáveis (frozen=True)	OK	Permite replace() seguro.
TypedDicts para dados brutos	OK	EditProductRawData; falta NewProductRawData.
Docstrings padronizadas	OK	Estilo uniforme em toda a base.
Dead code / imports não utilizados	OK	Zero após a limpeza.
FormField wrapper com propósito único	OK	Só get() e clear().
Fluxo de listar produtos (alertas)	OK	Estável.
Fluxo de abrir modo edição	OK	Preenche o form corretamente.
Fluxo de salvar edição	PENDENTE	update_product stub (1.3).
Fluxo de salvar novo produto	PENDENTE	_on_click_save_product stub (1.2) + 1.1.
Fluxo de deletar produto	BUG LATENTE	None > 0 (2.1).

Como usar este relatório

- Comece pela **Ordem de ataque** (página 1) — os itens 1.1, 1.2 e 1.3 destravam o app fim-a-fim.
- Cada item traz **Onde** (arquivo + linha), **Problema** e **Direção sugerida**.
- Prioridade 3 pode ser feita *oportunisticamente*, sempre que você já estiver no arquivo por outro motivo.
- Reavalie o backlog ao terminar cada prioridade — novas pendências vão surgir com a implementação.